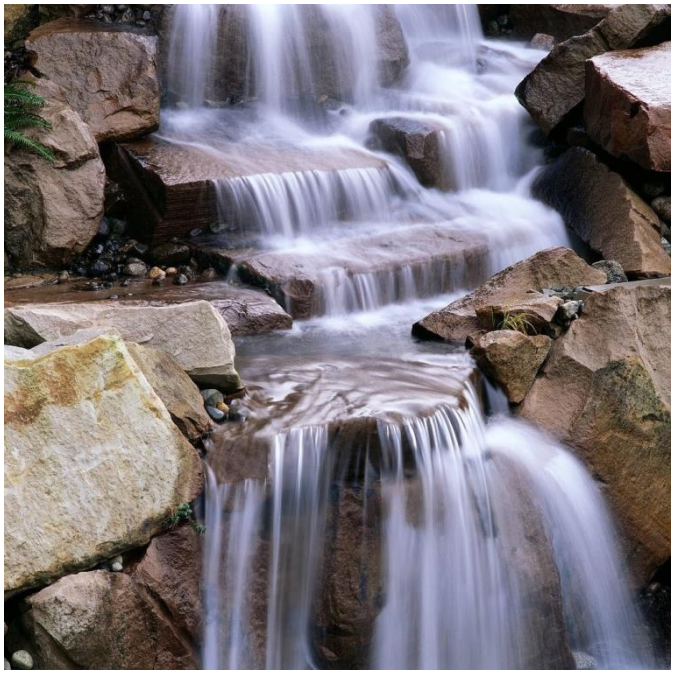




Maximum Flow – part 1



Reading:

CLRS

chapter 27 (1st Ed.)
chapter 26 (2nd and 3rd Ed.)

Maximum Flow

Input: 1. A directed graph (network) G=(V,E).

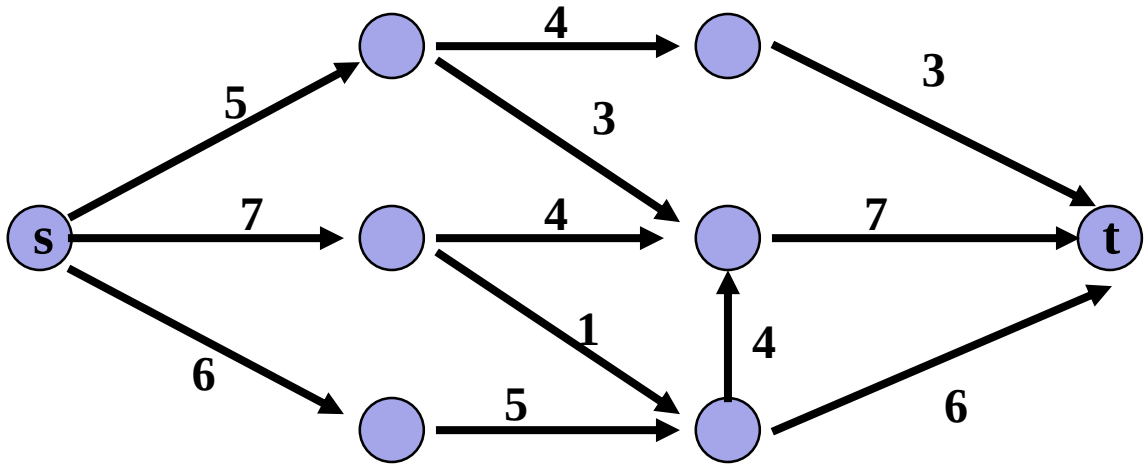
2. A function $c:E \to \mathbb{R}^{\geq 0}$. c(e) is the capacity of edge e.

3. A specified source node s and target node t.

Output: a flow function $f:E \to \mathbb{R}^{\geq 0}$, where f(e) is the amount of flow on edge e.

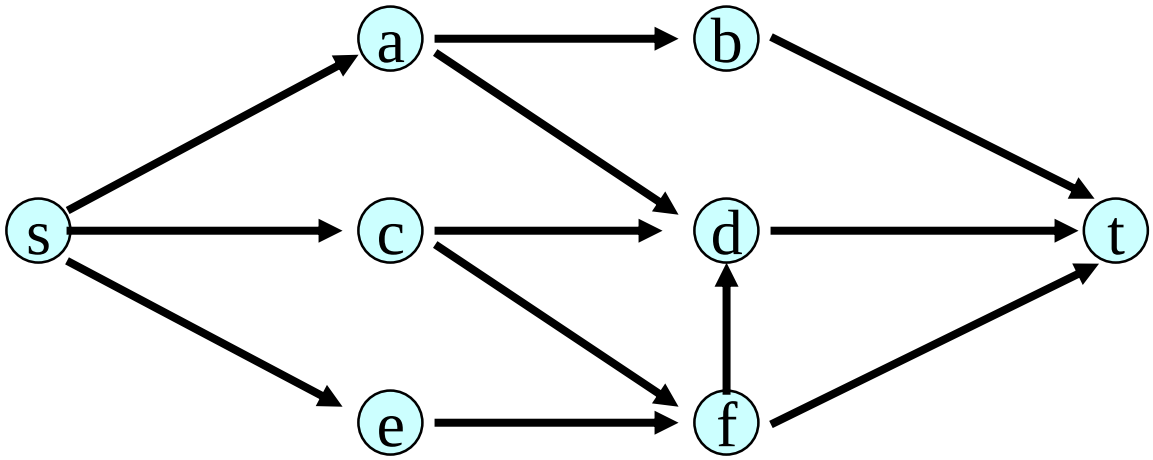
Goal: maximize the flow routed from s to t.

Input:



Maximum Flow

For a vertex v, let in(v) be the set of edges entering v, and let out(v) be the set of edges leaving v.



in(a)={(s,a)}, out(a)={(a,b),(a,d)},

in(d)={(a,d),(c,d),(f,d)}, out(d)={(d,t)}

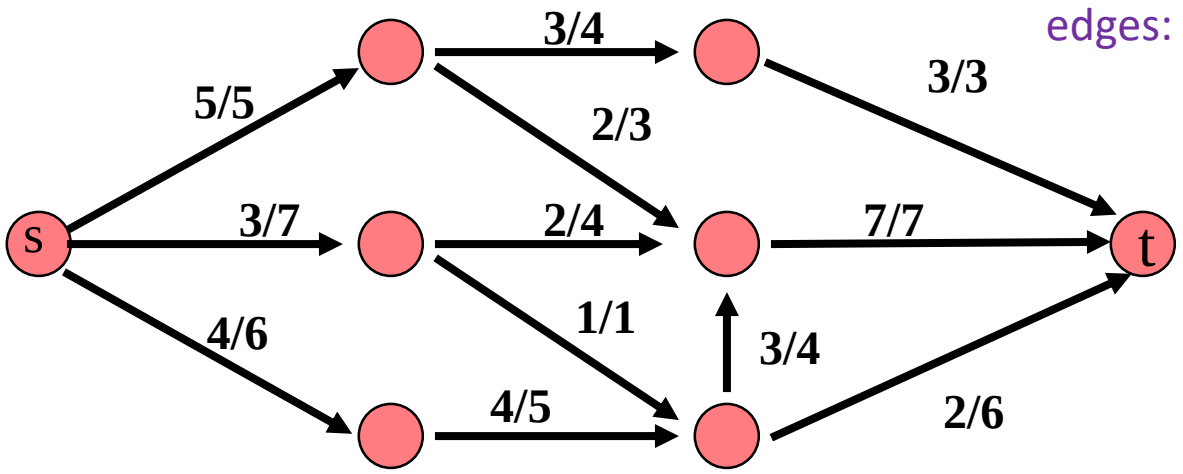
in(s)=∅, out(s)= {(s,a),(s,c),(s,e)}

A legal flow function
f(v,u) - flow on edge (v,u)

- Edge condition: $0 \leq f(u,v) \leq c(u,v)$: the flow through an edge cannot exceed the capacity of an edge.
- Vertex condition: for all v except s,t :
  $\Sigma_{e \in in(v)} f(e) = \Sigma_{e \in out(v)} f(e)$: the total flow entering a vertex is equal to total flow exiting this vertex.

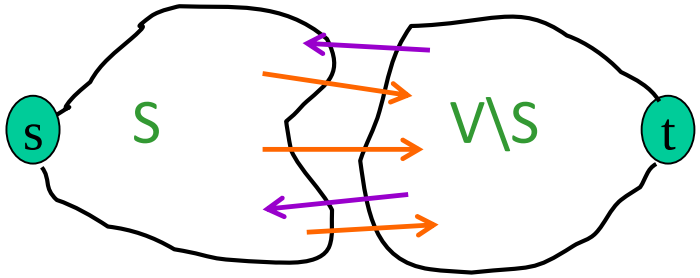
A legal flow:

Notation on edges: f(e)/c(e)



s-t cut in a directed network

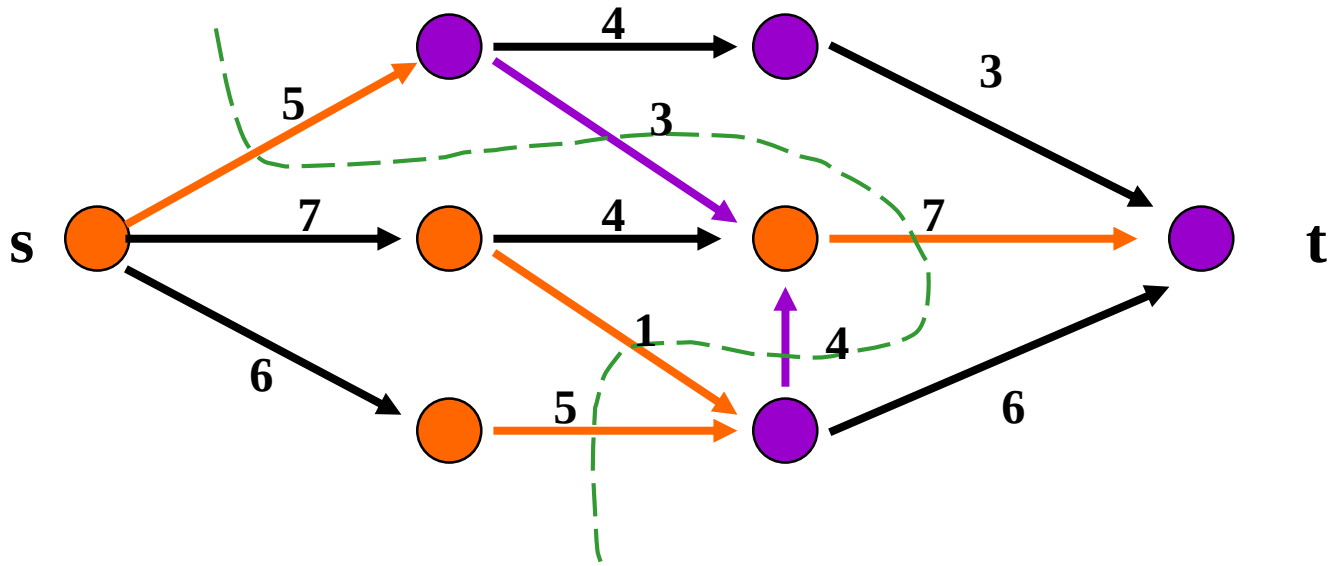
- An s-t cut (S,V\S) is a partition of the vertices, such that: (i) the source s is in S (ii) the target t is not in S. (iii) s is a root in the graph induced by S.
- The cut-edges: the set of edges (u,v) such that $u \in S$ and $v \notin S$, or $v \in S$ and $u \notin S$. We say that these edges cross the cut



out(S) – edges crossing the cut from S to V\S

in(S) – edges crossing the cut from V\S to S

Example of an s-t cut



S – set of orange vertices

out(S) – orange edges

in(S) – purple edges

The total capacity of edges in out(S) is 18

Value of a Flow

A flow function f is an assignment of a real number f(e) to each edge e such that the edge and vertex conditions hold for all the vertices/edges.

Definition: The value of the flow is $F = \sum_{e \in out(s)} f(e) - \sum_{e \in in(s)} f(e)$.

Claim: For any s-t cut (S,V\S): $F = \sum_{e \in out(S)} f(e) - \sum_{e \in in(S)} f(e)$.

Proof: soon.

In particular, for S=V\{t}: $F = \sum_{e \in in(t)} f(e) - \sum_{e \in out(t)} f(e)$.

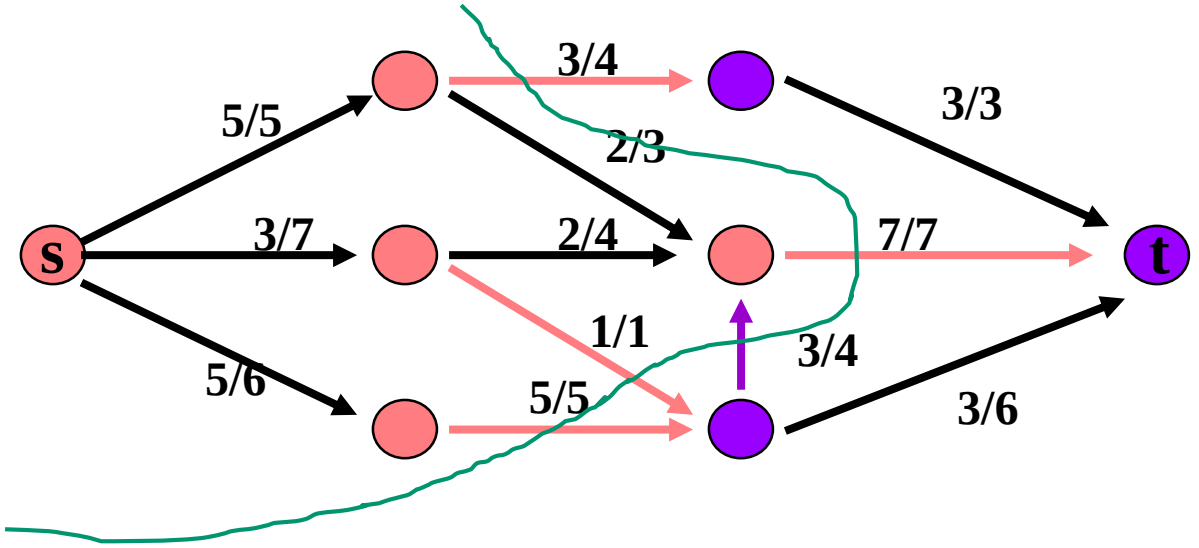
(so the flow leaving s = the flow entering t)

The flow entering s or leaving t is usually 0

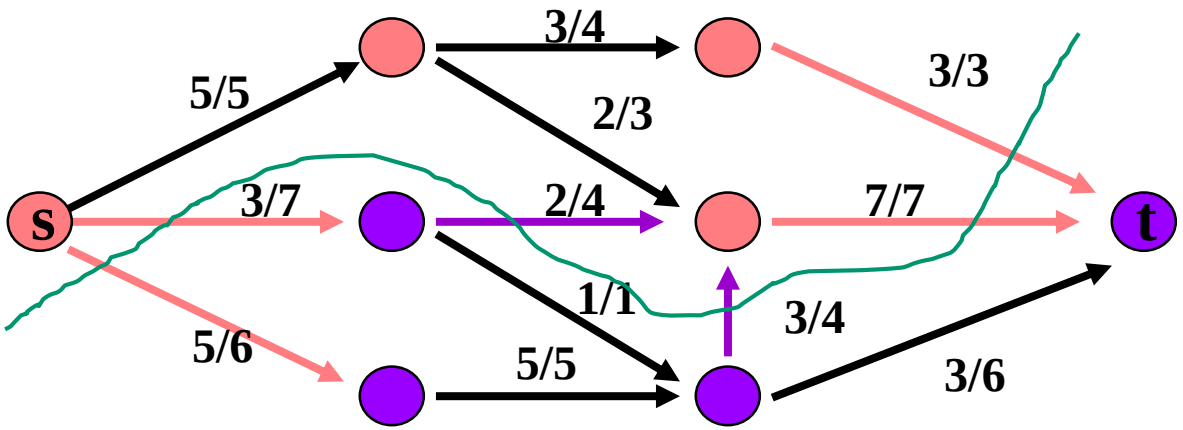
Example of the claim

F=5+3+5=13

Flow on out(S) = 3+5+1+7=16.
Flow on in(S)=3.
16-3=13



Flow on out(S) = 3+5+3+7=18.
Flow on in(S) = 2+3=5.
18-5=13



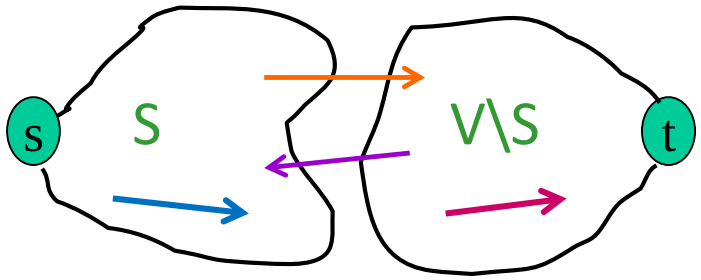
Value of a Flow

Proof: By definition, for the source s, $F = \sum_{e \in out(s)} f(e) - \sum_{e \in in(s)} f(e)$.

Also, for every vertex $v \in S\text{-}\{s\}$, $0 = \sum_{e \in out(v)} f(e) - \sum_{e \in in(v)} f(e)$.

Summing up, $F = \sum_{v \in S} (\sum_{e \in out(v)} f(e) - \sum_{e \in in(v)} f(e))$

In order to see what happens on the right-hand side of this equation, let's consider how the graph edges contribute to the sum in the r.h.s.

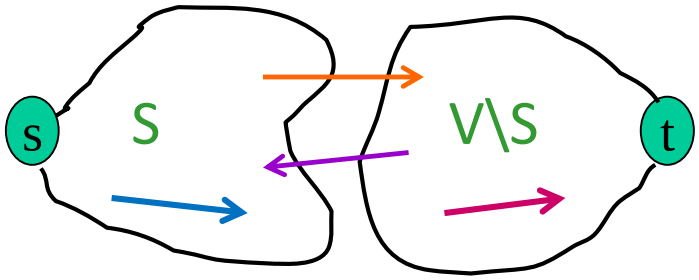


There are four classes of edges.

$$F = \sum_{v \in S} \left(\sum_{e \in out(v)} f(e) - \sum_{e \in in(v)} f(e) \right)$$

Consider an edge (x,y)

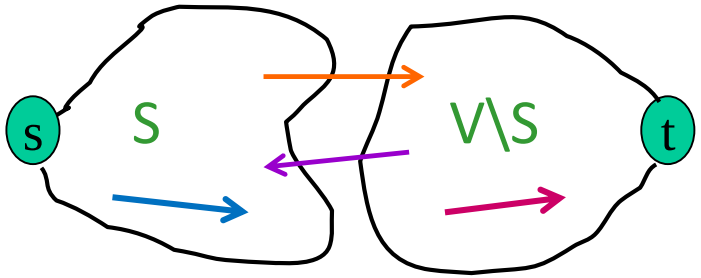
1. If $x \notin S$ and $y \notin S$ then f(e) does not appear on the r.h.s. at all.
2. If $x \in S$ and $y \in S$, then f(e) appears on the r.h.s. once positively, in the equation for x, and once negatively, in the equation for y. Thus, in the summation it is canceled out.
3. If $x \in S$ and $y \notin S$, that is $e \in out(S)$, then f(e) appears positively on the r.h.s. of the equation for x.
4. If $x \notin S$ and $y \in S$, that is $e \in in(S)$, then f(e) appears negatively on the r.h.s. of the equation for y.



Value of a Flow

Altogether, the r.h.s consists of the flow on edges in out(S) minus the flow on edges in in(S), as required.

Formally, for every cut S, $F = \sum_{e \in out(S)} f(e) - \sum_{e \in in(S)} f(e)$.



Capacity of an s-t cut

For an s-t cut (S,V\S), the capacity of the cut is

$$c(S) = \sum_{e \in out(S)} c(e).$$

Claim: For every flow function f with total flow F, and every s-t cut (S,V\S), $F \leq c(S)$.

Proof: We know that $F = \sum_{e \in out(S)} f(e) - \sum_{e \in in(S)} f(e).$

By the edge condition, $0 \leq f(e) \leq c(e), \forall e \in E$. Thus,

$$F \leq \sum_{e \in out(S)} c(e) - 0 = c(S).$$

Max-flow Min-Cut Theorem

The value of a maximum flow in a network is equal to the minimum capacity of an (s-t) cut.

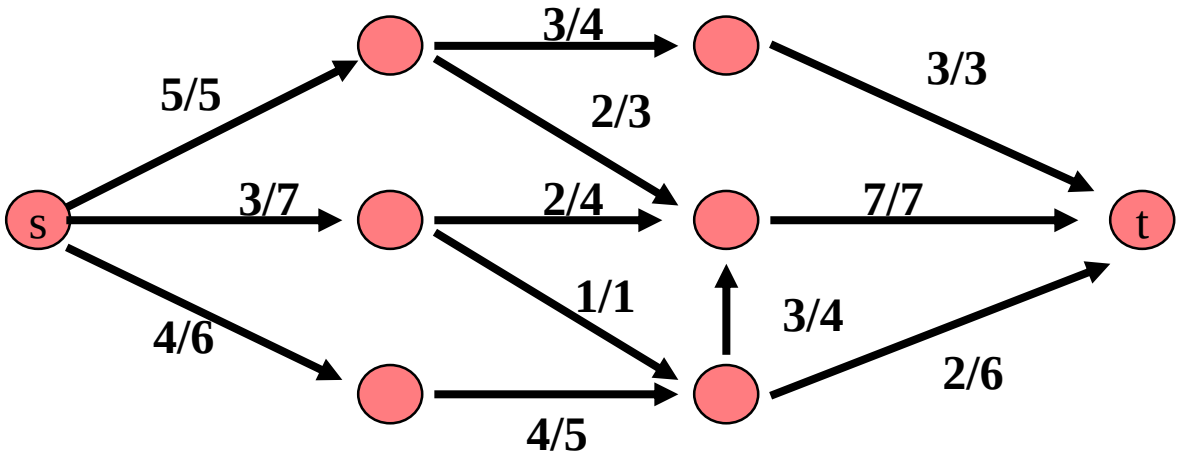
Proof:

max flow $\leq$ min cut: follows from the previous lemma.

max flow $\geq$ min cut: we will see an algorithm that produces a flow in which some cut is saturated.

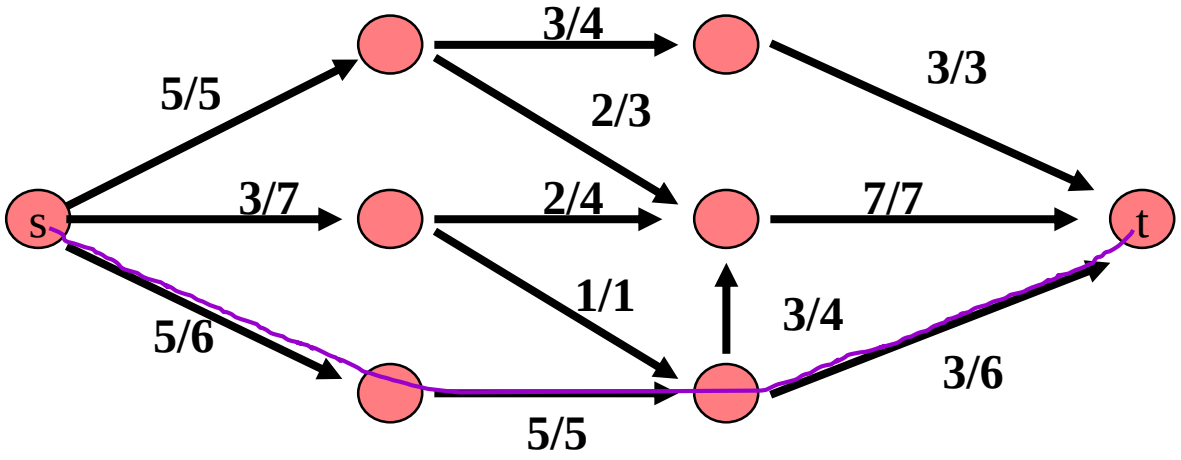
Discussion

Is this a max-flow?



Discussion

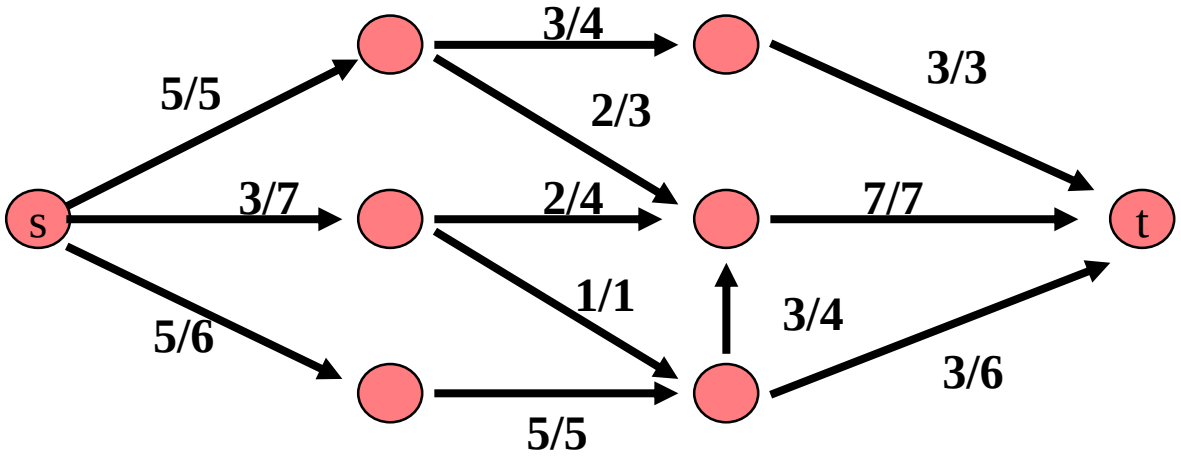
Is this a max-flow?



No – we can increase the flow along the lower s-t path

Discussion

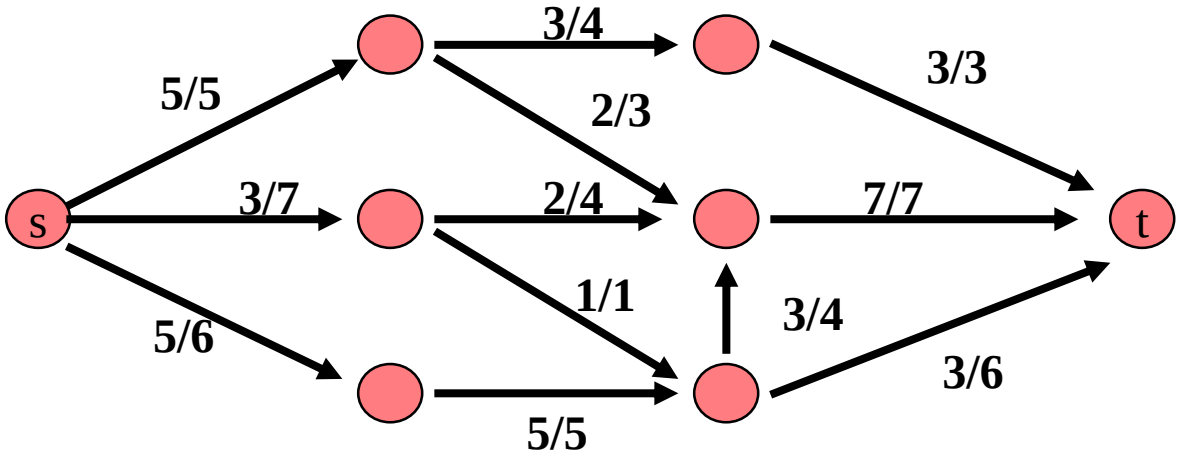
And now? Is this a max-flow?



It seems that no additional flow unit can be routed. ☹

Discussion

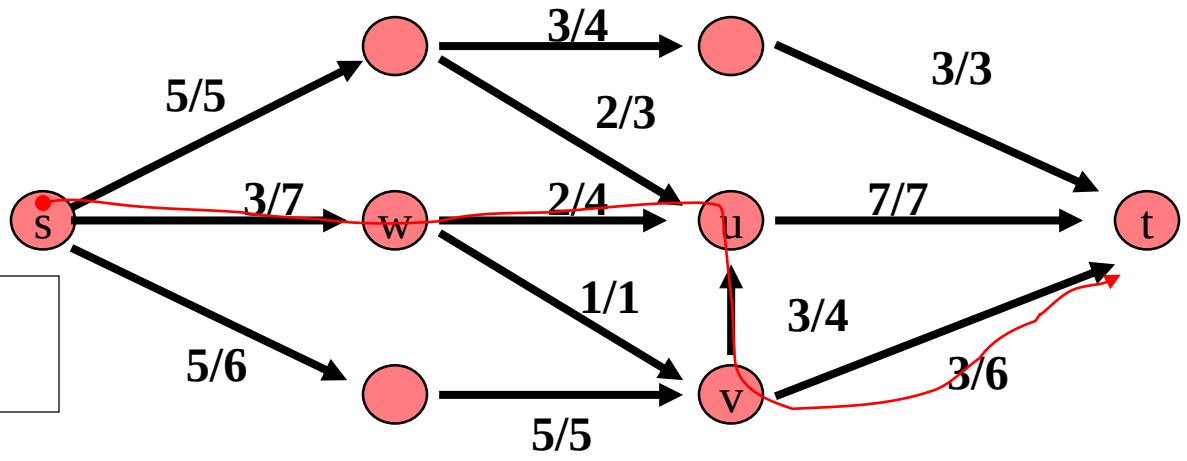
Right, no additional flow unit can be routed.
But… we can also take back flow.



An augmenting path with respect to a given flow f:

A directed path from s to t which consists of edges from G, but **not necessarily in the original direction**. Each edge on the path is either:

- forward edge: (w,u) in same direction as G and f(w,u) < c(w,u). (c(w,u)-f(w,u) is called *slack*) $\Rightarrow$ has room for more flow.
- backward edge: (u,v) in opposite direction in G (i.e., (v,u) in E) and f(v,u) > 0 $\Rightarrow$ we can 'take back' flow from such an edge.

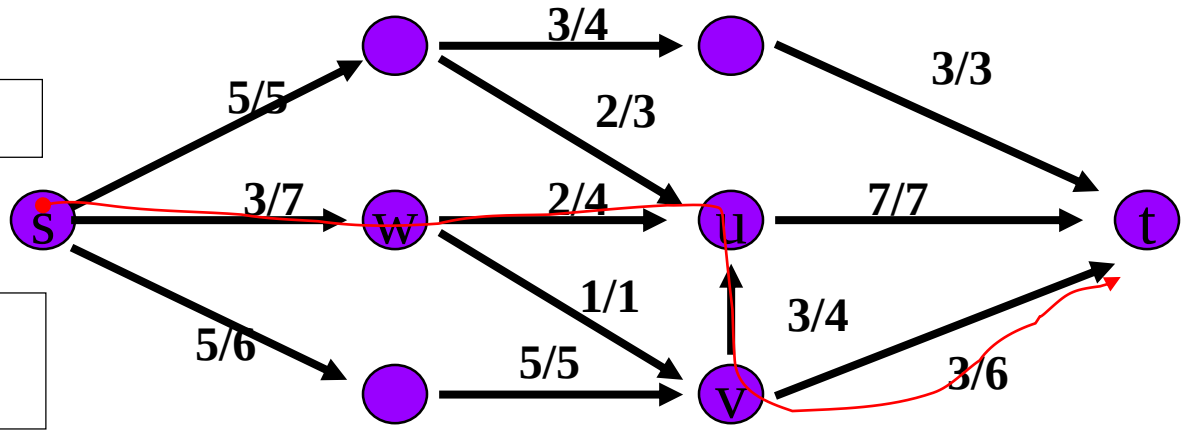


augmenting path s-w-u-v-t

An augmenting path with respect to a given flow f:

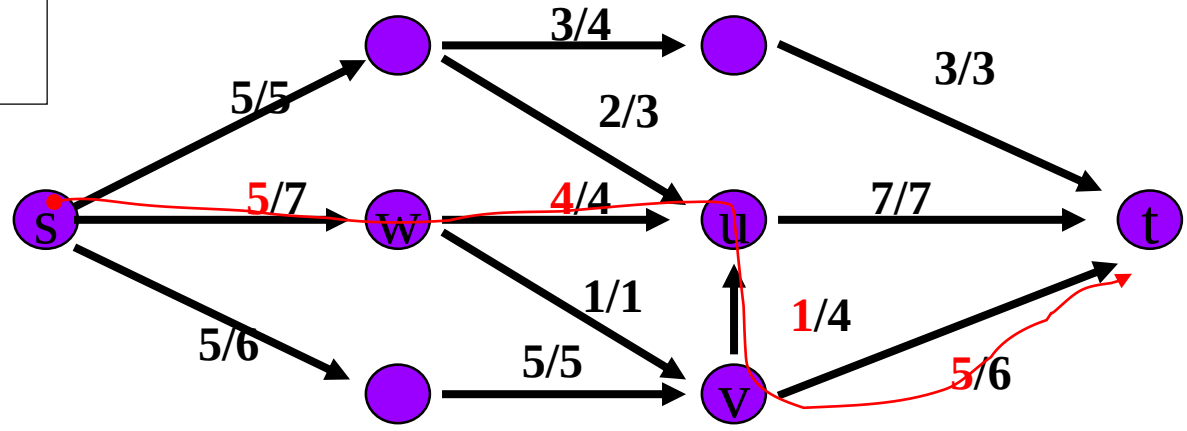
Flow value =13

augmenting path
s-w-u-v-t



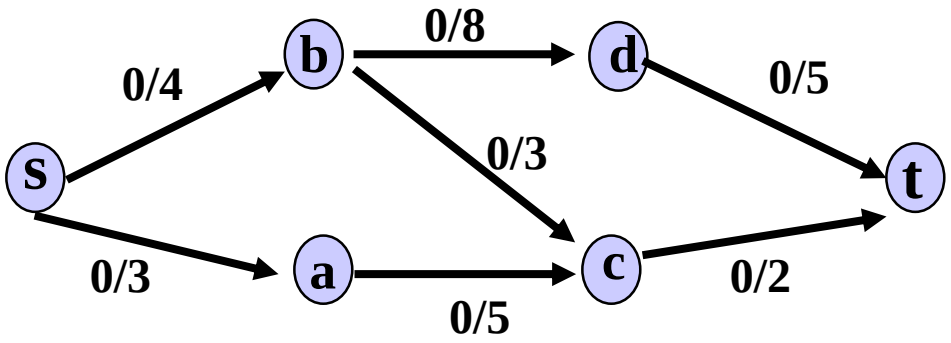
New flow.

Value = 15

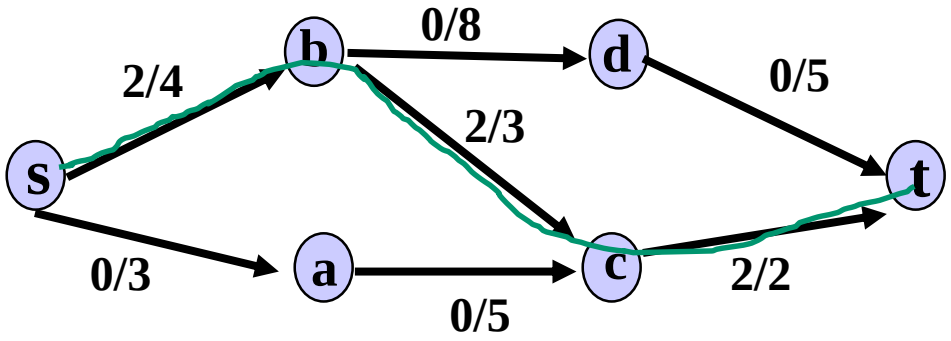


Another example:
Let's try improving the flow greedily

- Initial flow: f(e)=0 for all edges.

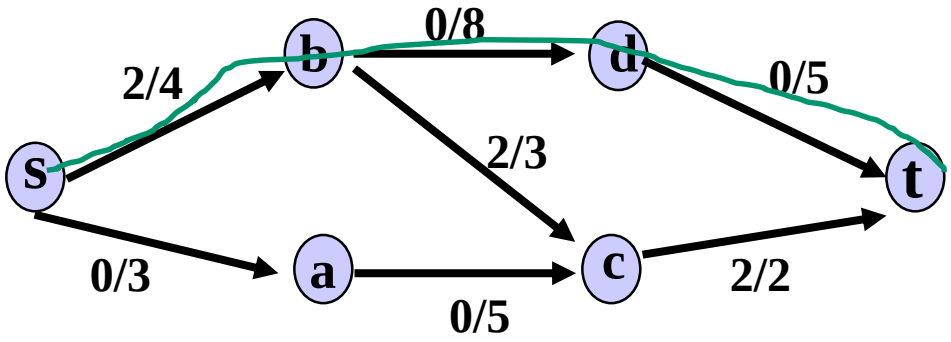


- Add 2 units along the path s-b-c-t:

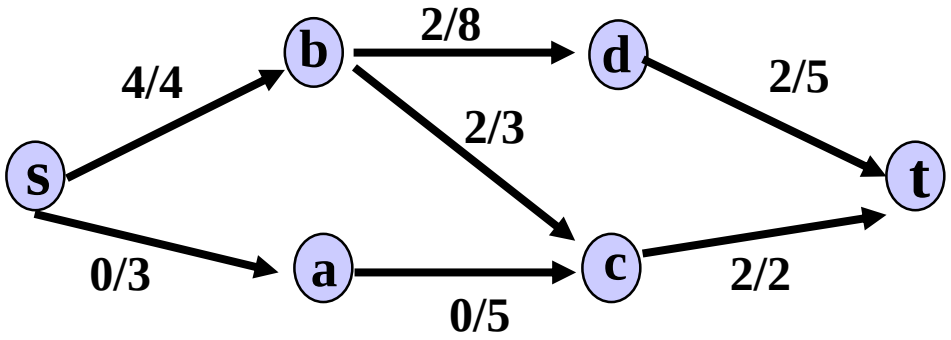


Let's try improving the flow greedily

- Next, add 2 units along the path s-b-d-t



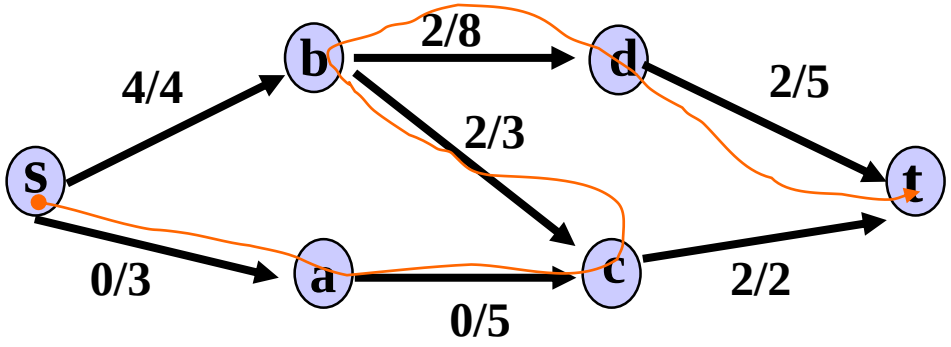
- There is no way to improve the flow greedily



But- we can use an augmenting path that includes backward edges

Using an augmenting path to increase flow

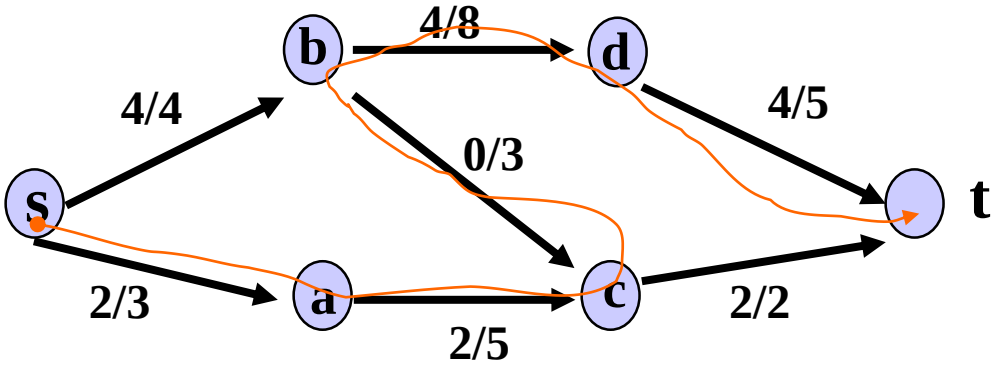
Push flow forward on forward edges, deduct flow from backward edges.



p = s-a-c-b-d-t

The amount of flow we can push:

minimum{ slacks along the forward edges on the path
flow along the backward edges on the path



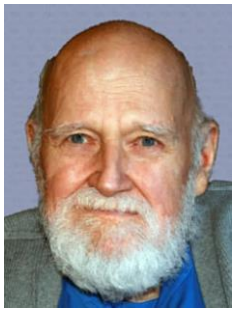
$\delta = 2$

The Ford-Fulkerson Method (1956)

- Initialize flow on all edges to 0.
- While there is an augmenting path, improve the flow along this path.

To implement FF, we need a way to detect augmenting paths.

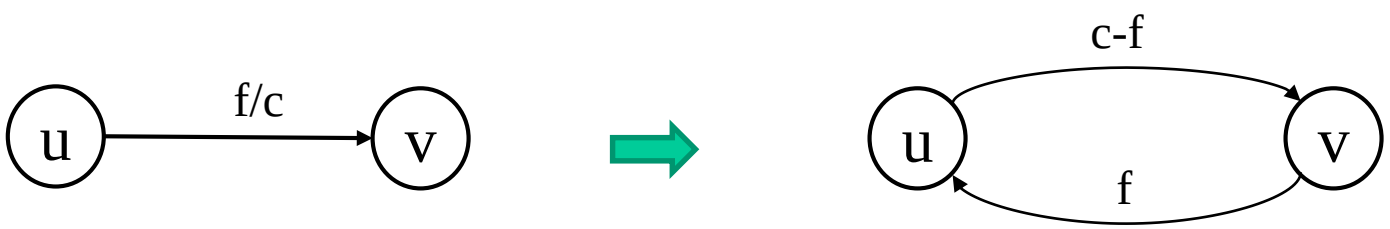
We build a residual graph with respect to the current flow.





Residual Graph w.r.t. flow f

- Given f, we build the residual graph: a network flow $R_f=(V,E')$
- An edge $(v,w)\in E'$ if either
  - (v,w) is a forward edge, and then its capacity in R_f is c(v,w)-f(v,w)
  - or (v,w) is a backward edge (that is, (w,v) is an edge with positive flow in G) , and then its capacity in R_f is f(w,v).
- An augmenting path is a regular directed path from s to t in R_f.



Ford-Fulkerson Method (G,s,t)

Initialize flow: $f(e)=0$ for every $e \in E$.

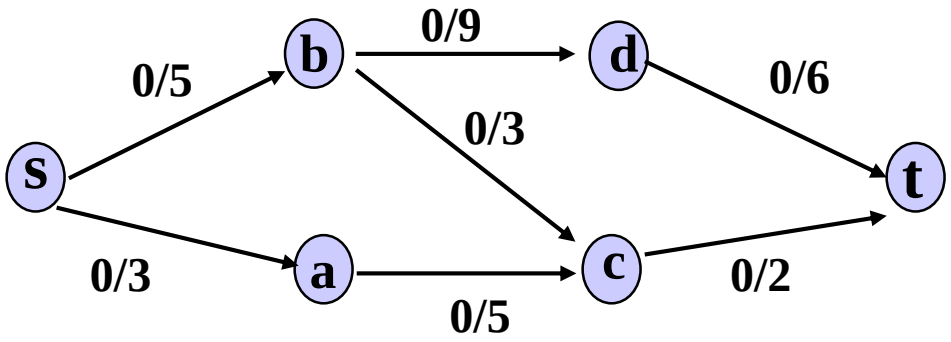
Repeat:

1. Build the residual network R_f with respect to the current flow f.
2. Find a path p from s to t in R_f.
3. Let δ = minimum capacity along p in R_f.
4. Augment δ units of flow along p and update f.

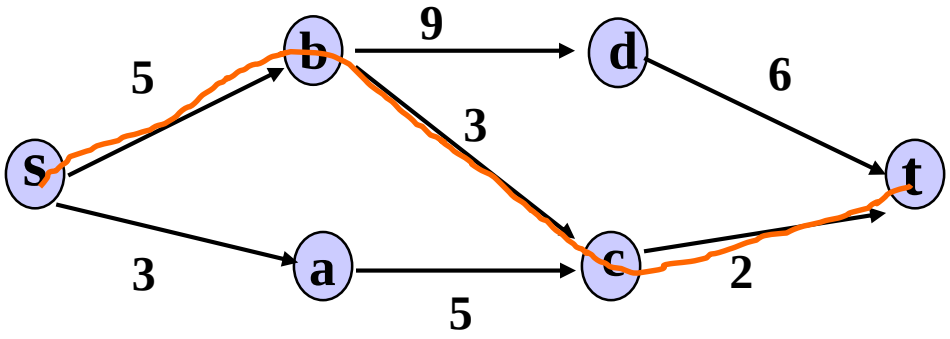
Until there is no path from s to t in R_f.

Ford-Fulkerson Method. Example (1)

- A given network, with initial flow f(e)=0 for every e∈E.



Note: in the first iteration R=G.

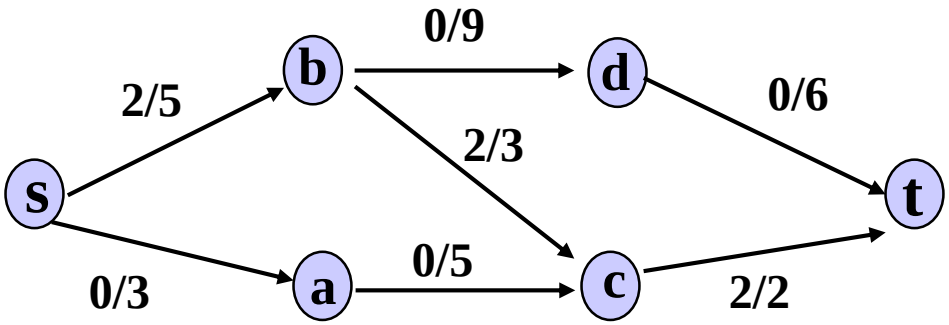


First augmenting path: s → b → c → t

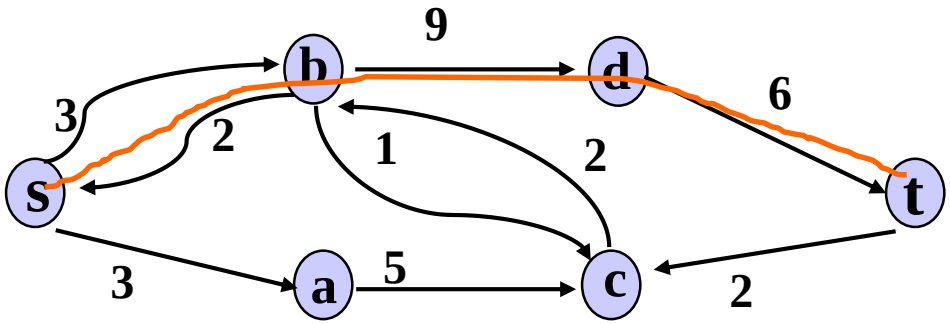
δ =2

Ford-Fulkerson Method. Example (2)

- The flow after one iteration:



- The residual network:

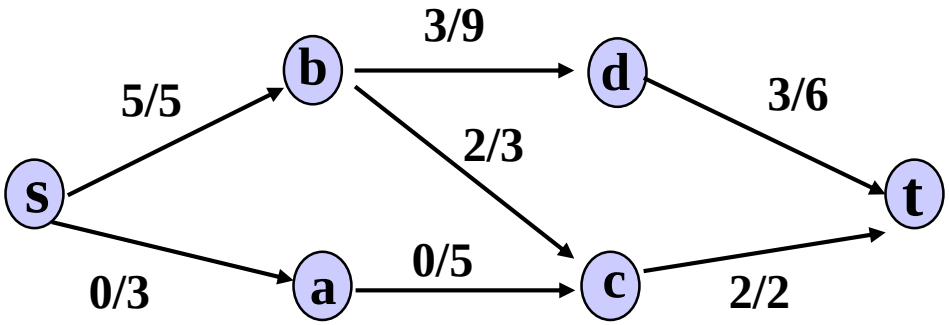


Second augmenting path: $s \rightarrow b \rightarrow d \rightarrow t$

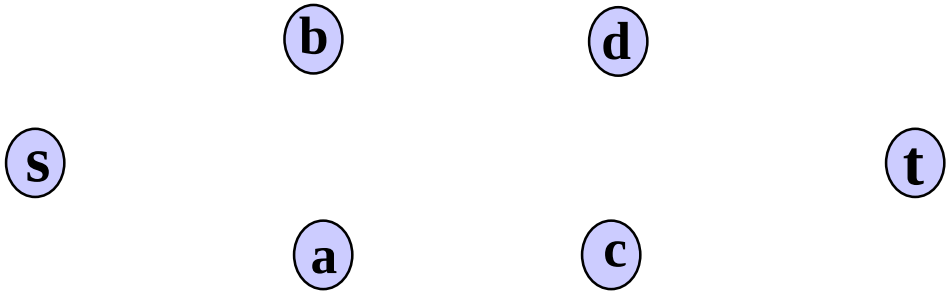
$\delta = 3$

Ford-Fulkerson Method. Example (3)

- The flow after two iterations:



- The residual network (complete in class):

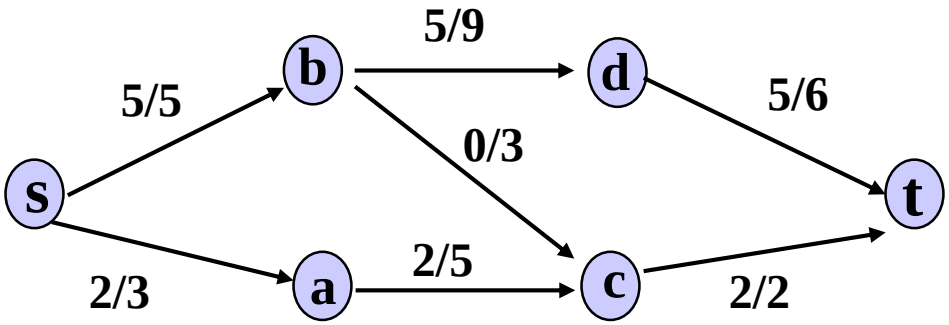


Third augmenting path: $s \rightarrow a \rightarrow c \rightarrow b \rightarrow d \rightarrow t$

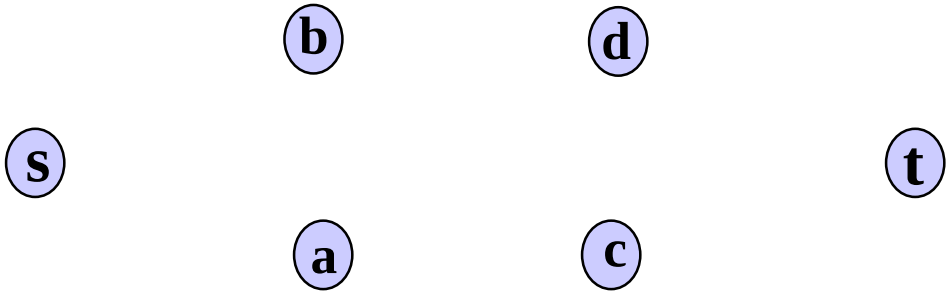
$\delta = 2$

Ford-Fulkerson Method. Example (4)

- The flow after three iterations:



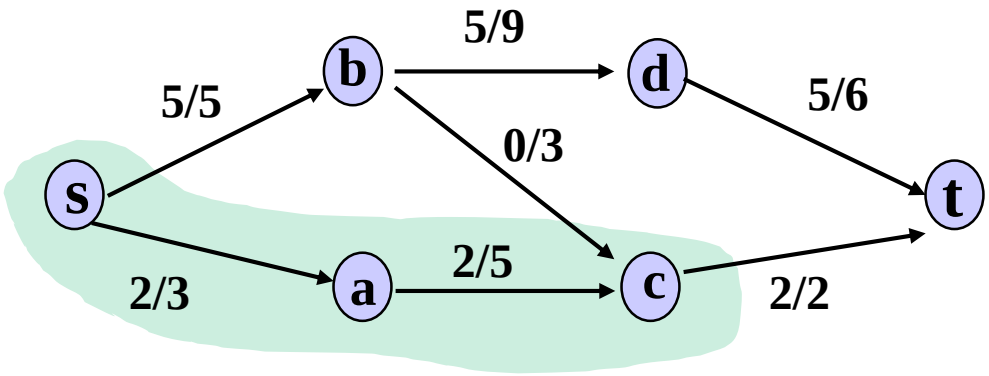
- The residual network (complete in class):



- There are no paths connecting s and t in the residual network!

Ford-Fulkerson Method. Example (5)

- Final flow: S={s,a,c} is a saturated min-cut.



Flow value = 7.

Note: All edges leaving {s,a,c} are saturated.

All edges entering {s,a,c} have flow 0.

Proof of Ford-Fulkerson Method.

Claim: The flow after each iteration is legal

Proof: The initial assignment (of $f(e)=0$ for all e) is clearly legal.

Let p be an augmenting path. Let δ be the minimum capacity along p in R_f. We show that the flow is legal after applying p.

Note: this is an induction on the number of iteration.

Egde condition: preserved by the selection of δ

Proof of Ford-Fulkerson Method.

Vertex condition: For each $v \notin p$, the flow passing through v does not change.

For each $v \in p$ $(v \neq s,t)$:

If a forward edge enters v and a forward edge leaves v, the flow on these two edges is increased by δ. The value of $in_f(v)$-$out_f(v)$ remains 0.

If a forward edge enters v and a backward edge leaves v, both edges enter v in G, δ flow-units change their entry edge to v, but the total entering flow remains the same.

The other two cases are similar (backward-backward, or backward-forward).

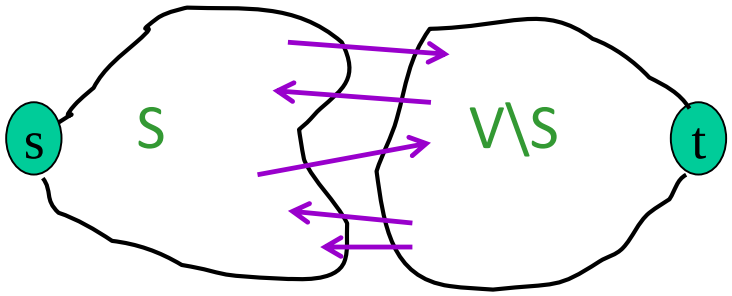
Proof of Ford-Fulkerson Method.

Theorem: A flow f is maximum iff it admits no augmenting path.

$\Rightarrow$ If an augmenting path exists, then the flow can be improved and is clearly not maximum.

$\Leftarrow$ Suppose f admits no augmenting path. We need to show that f is maximum.

We use the min-cut max-flow theorem: we will see (next slide) that when no augmenting path exists, some cut is saturated.



Proof of Ford-Fulkerson Method.

Let S be the set of vertices such that $v \in S$ iff there is an augmenting path from s to v.

The set S defines a cut.

Claim 1: for all edges in OUT(S): f(v,w) = c(v,w).

Proof: if f(v,w) < c(v,w) then w should join S.

Claim 2: for all edges in IN(S): f(v,w) = 0.

Proof: if f(v,w) > 0 then v should join S (backward edge).

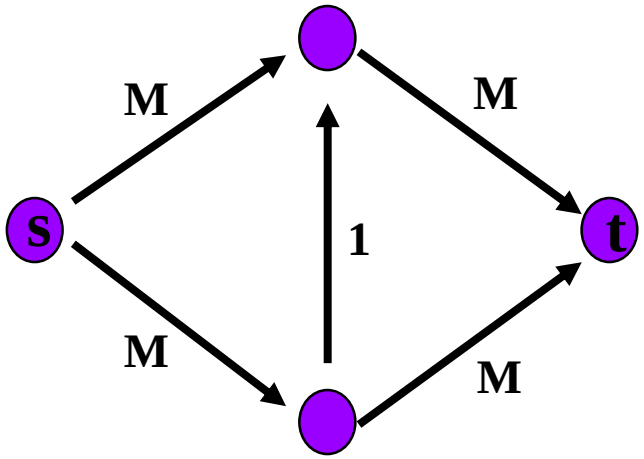
Therefore: The value of the flow is the capacity of the cut (S, V-S). Therefore, by the min-cut theorem, f is maximum.

Running Time of Ford-Fulkerson

Each iteration (building R_f and detecting an augmenting path) takes $O(|E|)$ (how?).

How many iterations are there?

If all capacities are integers, then in every iteration, $\delta \geq 1$. The number of iterations is at most f^* when f^* is the value of the maximum flow.





The time complexity of FF is $O(|E|f^*)$, when f^* is the value of the maximum flow.

Running Time of Ford-Fulkerson

- Can we bound the number of iterations if the capacities are not integer?

  Unfortunately, no. There exists a flow network with real capacities such that Ford-Fulkerson does not terminate.

  Moreover, the value of the flow converges to a value arbitrarily far from the max flow.



- On the other hand, this issue can be solved by a wise selection of augmenting path.



Edmonds-Karp Algorithm:

Use FF method. Search for augmenting path using breadth-first search, i.e., the augmenting path is always a **shortest path** (minimal number of edges) from s to t in the residual network.

Theorem (proof omitted): This way, the number of augmentations is $O(|V||E|)$.

The resulting complexity: $O(|V||E|^2)$

(each iteration takes $O(|E|)$)

Greedy augmenting path Selection

Use FF method. In each iteration select an augmenting path with the maximal δ value.

The time complexity of this algorithm is

$O(|E|^2 \log |E| \log f^*)$.

Explanation (proof omitted): The widest augmenting path can be computed in time $O(|E| \log |E|)$, and there will be at most $O(|E| \log f^*)$ iterations.